

Распознавание КС-грамматик. Программирование сканера

Теория автоматов и формальных языков

Лекция №10

Структура лекции

- Программирование сканера.
- Организация таблиц символов.
- Хеш-адресация.
- Способы решения задачи коллизии.
- Рехеширование.
- Хеш-функции.
- Контекстно-свободные грамматики.
- Синтаксически управляемый перевод.
- Окончание работы алгоритма.

Программирование сканера

Необходимо построить лексический анализатор для выделения из входного потока исходных составляющих языка – лексем или символов. Символами являются:

1. разделители или операторы (+, -, *, /, (,) и т.п.);
2. служебные слова (что-то вроде begin, end, ... и т.п.);
3. идентификаторы;
4. числа.

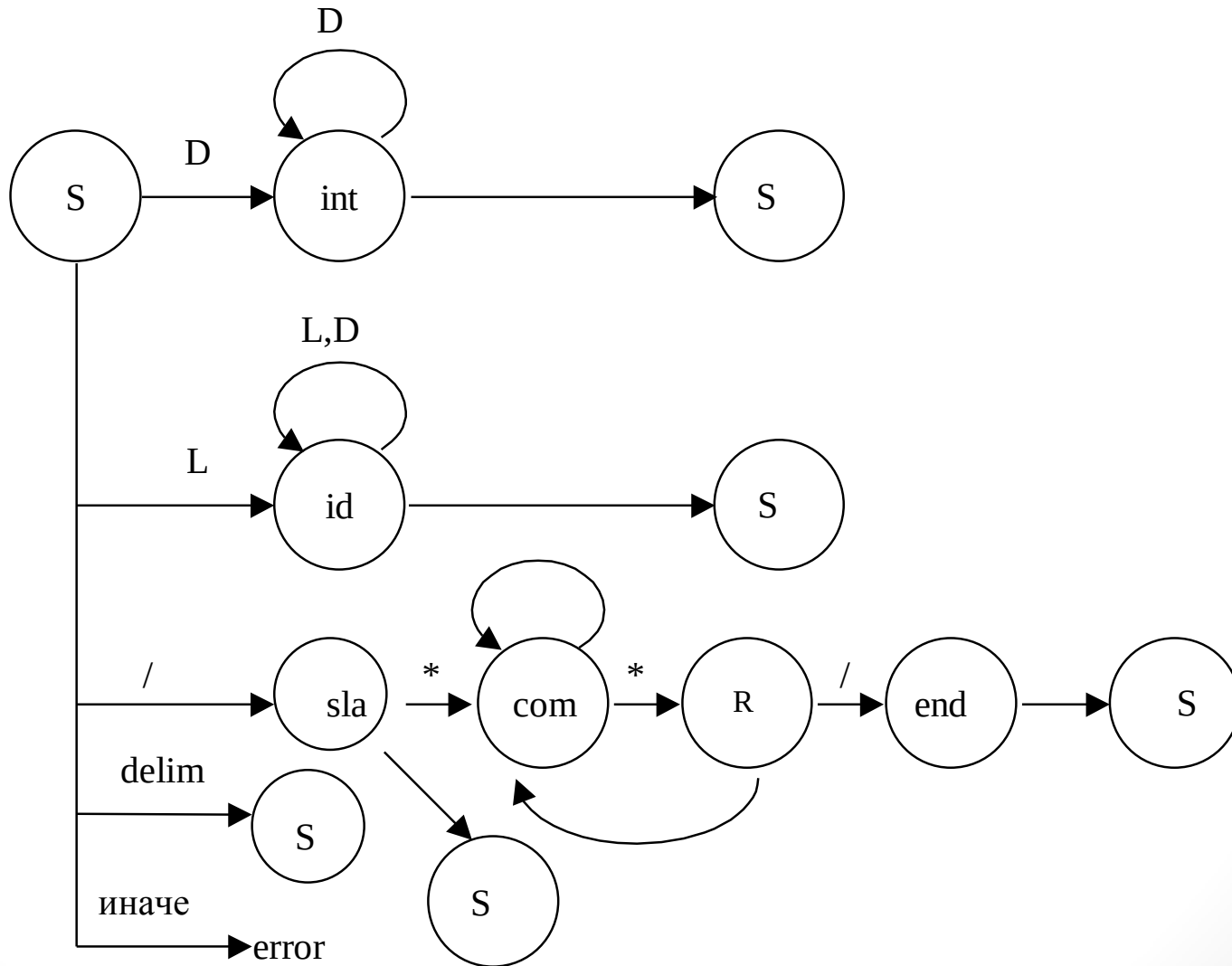
Кроме того, необходимо исключать комментарии (как строчные, типа '//', так и многострочные, вида '/*...*/').

Для построения сканера используется конечный автомат.

К автомату предъявляются следующие минимальные требования:

1. Автомат должен формировать поток лексем;
2. Автомат должен заносить полученные лексемы в таблицу символов или имен.

Структура автомата



Особенности автомата

Следует отметить, что

1. При переходе в начальное состояние S иногда требуется возвращать обратно считываемый символ входной последовательности.
2. Автомат в таком виде бесполезен, он не выполняет никаких полезных процедур.

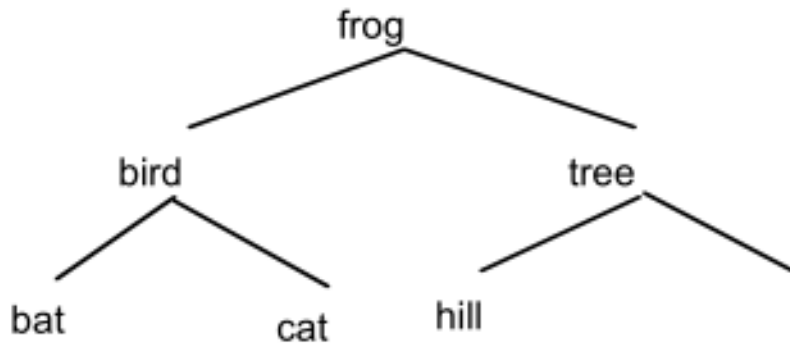
Автомат необходимо дополнить **процедурной** частью.

Организация таблиц символов

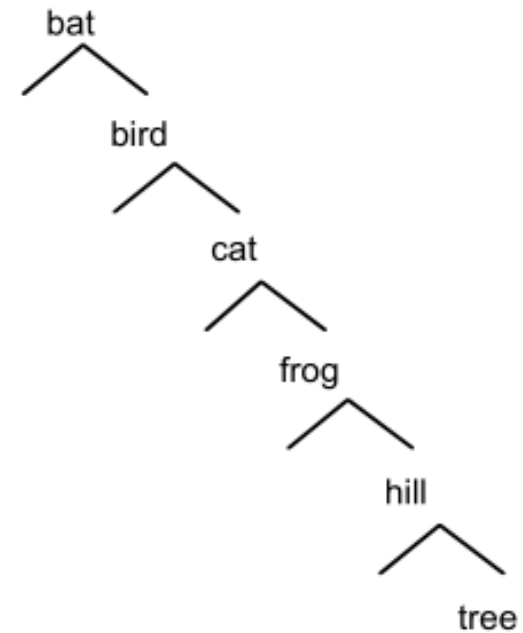
- Простейший способ организации – использовать неупорядоченную таблицу.
- Новый элемент добавляется в таблицу в порядке поступления.
- Среднее время поиска пропорционально $n/2$.

Способы хранения таблиц.

Бинарное дерево



(а) Сбалансированное бинарное дерево



(б) Несбалансированное бинарное дерево

Хеш-адресация

Хеш-адресация – это метод преобразования имени в индекс элемента в таблице.

- Если таблица состоит из N элементов, то им присваиваются номера $0, 1, \dots, N-1$.

Тогда назовем **хеш-функцией** некоторое преобразование имени в адрес.

- Простейший вариант хеш-функции – это использование внутреннего представления первой литеры имени.

Способы решения задачи КОЛЛИЗИИ.

Пусть хешируется имя S и при этом обнаруживается коллизия.

- Обозначим через h значение хеш-функции. Тогда сравниваем S с элементом по адресу $(h+p_1) \bmod N$, где N – длина таблицы (максимальное число элементов в таблице).
- Если опять возникает коллизия, то выбираем для сравнения элемент с адресом $(h+p_2) \bmod N$ и т.д.

Рехеширование

- Способы **рехеширования** заключаются в выборе чисел $p_1, p_2, \dots, p_n$. Рассмотрим наиболее распространенные из них.
- Для начала введем параметр, называемый *коэффициентом загрузки таблицы* lf :

$$lf = n/N, \text{ где}$$

n – текущее количество элементов в таблице,

N – максимально возможное число элементов.

Способы рехеширования

Линейное рехеширование

- $p_i = i$ (т.е. $p_1=1, p_2=2, \dots, p_n=n$).
- Среднее число сравнений E при коэффициенте загрузки lf определяется как
- $E(lf) = (1-lf/2)/(1-lf)$.

Рехеширование сложением

- $p_i = i(h+1)$, где h – исходный хеш-индекс.

Случайное рехеширование

Случайное рехеширование

Пусть максимальное число элементов в таблице кратно степени двойки, т.е. $N=2^k$. Вычисление p_i осуществляется по следующему алгоритму:

1. $R := 1$.
2. Вычисляем p_i :
 1. $R := R * 5$
 2. выделяем младшие $k+2$ разрядов R и помещаем их в R
 3. $p_i := R \gg 1$ (сдвигаем R вправо на 1 разряд)
3. Перейти к 2.

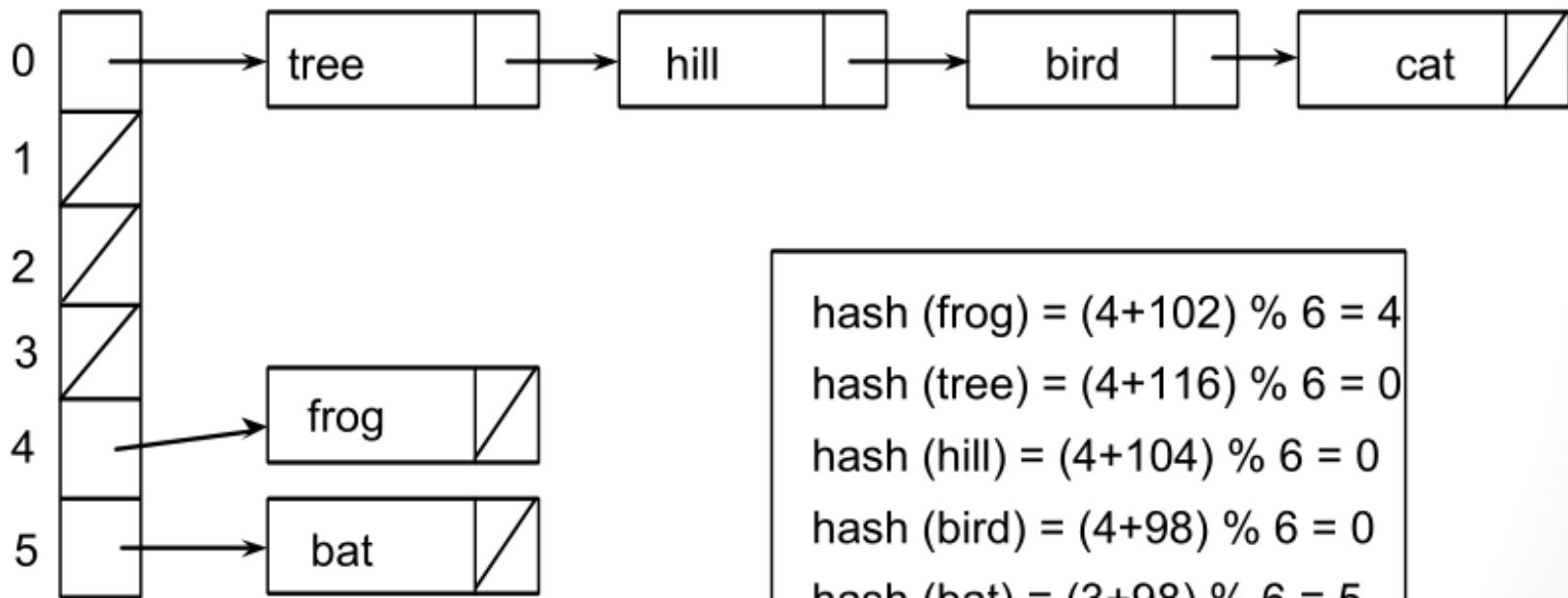
Хеш-функции

- Имя представляет множество литер (символов), и для вычисления индекса поступают обычно следующим способом:
 1. Берется S' , являющееся результатом суммирования литер из исходного символа S (либо простое суммирование, либо поразрядное исключающее ИЛИ):
 2. Вычисляется окончательный индекс. При этом возможны самые различные варианты. В частности:
 - Умножить S' на себя и использовать n средних битов в качестве значения h (для $N=2^n$);
 - Использовать какую-либо логическую операцию (например, исключающее ИЛИ) над некоторыми частями S' ;
 - Если $N=2^n$, то разбить S' на n частей и просуммировать их. Использовать n крайних правых битов результата;
 - Разделить S' на длину таблицы, и остаток использовать в качестве хеш-индекса.

Способы хранения таблиц.

Хеш-таблицы

- Хеш-таблицы могут быть организованы как массив или **массив связанных списков**.



$\text{hash}(\text{frog}) = (4 + 102) \% 6 = 4$
 $\text{hash}(\text{tree}) = (4 + 116) \% 6 = 0$
 $\text{hash}(\text{hill}) = (4 + 104) \% 6 = 0$
 $\text{hash}(\text{bird}) = (4 + 98) \% 6 = 0$
 $\text{hash}(\text{bat}) = (3 + 98) \% 6 = 5$
 $\text{hash}(\text{cat}) = (3 + 99) \% 6 = 0$

Способы хранения таблиц.

Хеш-таблицы

- Изначально список состоит из массива, элементы которого являются заголовками связанного списка.
- Слово, которое должно быть занесено в таблицу добавляется в один из списков. Для определения куда именно используется специальная **хеш-функция**.
- Хеш-функция принимает в качестве аргумента само слово и возвращает целочисленное значение – **номер соответствующего списка**. Поиск в каждом списке осуществляется последовательно до тех пор, пока слово не будет найдено в списке. Если слово не найдено, то оно помещается в конец списка.
- **Критичен выбор правильной хеш-функции.** Обычно выбирают хеш-функцию, которая является комбинацией арифметических выражений на базе символов, из которых состоят входные слова.

Например:

- сумма количества символов из которых состоит слово и `ascii`-кода первой буквы слова, и остаток от деления на `Hashmax`, где **Hashmax** – размер массива.
- Структура таблицы не будет зависеть от того, в каком порядке читаются слова. За исключением порядка слов в конкретном списке.

Контекстно-свободные грамматики

определение КСГ внешне выглядит как и обычное
определение грамматики: $G_{\text{КСГ}} = (N, \Sigma, P, S)$, однако при этом
характерным для КСГ является форма ее правил (продукций)

- $P: A \rightarrow \alpha,$

где $A \in N, \alpha \in (N \cup \Sigma)^+.$

Пример КС-грамматики

В качестве примера рассмотрим, как мог бы выглядеть некий командный, внешне приближенный к естественному язык:

$N = \{\text{группа_существ, глагол, прилагательное, существительное, предлог, фраза}\}$

$\Sigma = \{\text{печатать, стереть, зеленый, первый, последний, символ, строка, страница, в}\}$

$S = \text{фраза}$

$P = \{$

$\text{Фр} \rightarrow \Gamma, \Gamma\text{с}; \quad \Gamma\text{с} \rightarrow \text{Прил}, \text{С}; \quad \Gamma\text{с} \rightarrow \text{Прил}, \text{С}, \text{Предлог}, \Gamma\text{с};$

$\Gamma \rightarrow \text{печатать}; \quad \Gamma \rightarrow \text{стереть}; \quad \text{Прил} \rightarrow \text{зеленый};$

$\text{Прил} \rightarrow \text{первый}; \quad \text{Прил} \rightarrow \text{последний}; \quad \text{С} \rightarrow \text{символ};$

$\text{С} \rightarrow \text{строка}; \quad \text{С} \rightarrow \text{страница} \quad \text{Предлог} \rightarrow \text{в}\}$

ОК-грамматики

- Несмотря на кажущуюся общность КСГ, при описании естественно-подобных языков существуют ограничения, связанные с различного рода бесконтекстными согласованиями (рода, падежа, числа и т.п.).
- Для решения проблемы подобных согласований можно ввести еще одну разновидность грамматик – грамматики *определенных клауз*. Их особенностью является наличие *контекстуальных аргументов*.

Синтаксически управляемый перевод

Рассмотрим грамматику арифметического выражения.

- $S \rightarrow E$
- $E \rightarrow T$
- $E \rightarrow E + T$
- $T \rightarrow F$
- $T \rightarrow T * F$
- $F \rightarrow a$
- $F \rightarrow (E)$

где E-выражение, T-терм, F-формула, a-идентификатор.

Левый вывод цепочки

- Например, пусть дана фраза: $a+a^*a$. Воспользовавшись данной грамматикой, получим следующую цепочку вывода:
- $S \rightarrow E \rightarrow E+T \rightarrow E+T^*F \rightarrow T+T^*F \rightarrow F+F^*F \rightarrow a+a^*a$

Синтаксически управляемый перевод

- Применение каждого правила грамматики будет вызывать выполнение той или иной семантической процедуры. Именно в этом заключается основная идея синтаксически управляемого перевода.
- Рассмотрим схему перевода, отображающего арифметические выражения из языка $L(G_0)$ в соответствующие постфиксные польские записи.
- Согласно определению постфиксной формы записи, правилу $E \rightarrow E+T$ соответствует элемент перевода $E=ET+$

Перевод выражения в постфиксную форму записи

- При этом будем дописывать справа от полученной последовательности результат применения соответствующего элемента перевода:

$(E, E) \Rightarrow (E+T, ET+)$

$\Rightarrow (T+T, TF+)$

$\Rightarrow (F+T, FT+)$

$\Rightarrow (a+T, aT+)$

$\Rightarrow (a+T^*F, aTF^*+)$

$\Rightarrow (a+F^*F, aFF^*+)$

$\Rightarrow (a+a^*F, aaF^*+)$

$\Rightarrow (a+a^*a, aaa^*+).$

Правило	Элемент перевода
$E \rightarrow E+T$	$E = ET+$
$E \rightarrow T$	$E = T$
$T \rightarrow T^*F$	$T = TF^*$
$T \rightarrow F$	$T = F$
$F \rightarrow (E)$	$F = E$
$F \rightarrow a$	$F = a$

Перевод инфиксной формы записи в польскую

- более эффективным является следующий метод. Основная идея остается прежней – применение каждого правила грамматики влечет выполнение некоторых действий.
- Эти действия мы будем называть *семантическими программами* или *процедурами*.

Перевод инфиксной формы записи в польскую

Пусть задана следующая грамматика арифметического выражения:

- $Z ::= E$
- $E ::= T \mid E+T \mid E-T \mid -T$
- $T ::= F \mid T * F \mid T / F$
- $F ::= a \mid (E)$

Перевод инфиксной формы записи в польскую

- Схема перевода:

№	Правило	Семантическая программа
1	$Z ::= E$	нет (*)
2	$E ::= T$	нет (**)
3	$E ::= E+T$	Push('+') (**)
4	$E ::= E-T$	Push('-') (**)
5	$E ::= -T$	Push('@')
6	$T ::= F$	Нет
7	$T ::= T*F$	Push('*')
8	$T ::= T/F$	Push('/')
9	$F ::= a$	Push(a)
10	$F ::= (E)$	Нет

Правило (1) применимо, если $R=\#$ (символ # означает конец анализируемой последовательности);
правила (2), (3), (4) применимы, если в R содержится '+', '-', '#' или ')'.
Семантическая процедура *Push(X)* добавляет в конец выходной цепочки символ X.

Алгоритм СУ-перевода

- для того, чтобы реализовать алгоритм разбора без полного перебора возможных вариантов применимости правил, нам потребуется стек S и переменная R , которая будет хранить очередной считываемый символ.
- **Алгоритм СУ-перевода** выглядит так: сначала в стек S заносится символ $\#$. Далее к содержимому стека мы пробуем применить какое-либо правило из списка. Если ни одно из правил не срабатывает, то в стек заносится очередной символ анализируемой входной последовательности.

Синтаксически управляемый перевод

- Рассмотрим разбор выражения "a*(b+c)#":

Стек S	R	$\omega_k \dots$	Номер правила	Польская цепочка
#	a	*(b+c)#		
#a	*	(b+c)#	9	a
#F	*	(b+c)#	6	a
#T	*	(b+c)#		a
#T*	(	b+c)#		a
#T*(	b	+c)#		a
#T*(b	+	c)#	9	ab
#T*(F	+	c)#	6	ab
#T*(T	+	c)#	2	ab
#T*(E	+	c)#		ab
#T*(E+	c	)#		ab
#T*(E+c	)	#	9	abc
#T*(E+F	)	#	6	abc
#T*(E+T	)	#	3	abc+
#T*(E	)	#		abc+
#T*(E)	#		10	abc+
#T*F	#		7	abc+*
#T	#		2	abc+*
#E	#		1	abc+*
#Z	#		STOP	abc+*

№	Правило	Семантическая программа
1	$Z ::= E$	нет (*)
2	$E ::= T$	нет (**)
3	$E ::= E+T$	Push('+') (**)
4	$E ::= E-T$	Push('-') (**)
5	$E ::= -T$	Push('@')
6	$T ::= F$	Нет
7	$T ::= T*F$	Push('*')
8	$T ::= T/F$	Push('/')
9	$F ::= a$	Push(a)
10	$F ::= (E)$	Нет

Окончание работы алгоритма

- **Признак нормального окончания работы алгоритма:** когда в стеке остался единственный символ Z , а текущим символом является $\#$ - символ конца входной последовательности, то мы считаем, что процедура синтаксического анализа завершена успешно. В противном случае (если в стеке есть другие символы) фраза построена неверно.